2024 lEEE 13th Data Driven Control and Learning Systems Conference
May 17-19, 2024, Kaifeng, China

Improved Residual Reinforcement Learning for Dynamic Obstacle Avoidance of Robotic Arm

Zhenting Liu, Shan Liu

State Key Laboratory of Industrial Control Technology, College of Control Science and Engineering
Zhejiang University, Hangzhou 310027, China
E-mail: liuzhenting@zju.edu.cn, sliu@zju.edu.cn

Abstract: This paper proposes an improved residual deep reinforcement learning method for robot arm dynamic obstacle avoidance and position servo. The proposed method first simplifies the state space by constructing key points and pre-trains a model capable of completing obstacle avoidance tasks using the simplified state. Then, when training with real state information, a guiding network is used to help accumulate good samples, which improves the training efficiency. To overcome the convergence difficulty of residual DQN in robot arm obstacle avoidance, this paper incorporates the action of the feedback controller into the action space and uses incremental reward values to evaluate the actions. Simulation results demonstrate that the proposed method can effectively achieve robot arm dynamic obstacle avoidance and position servo.

Key Words: Robot arm, Deep reinforcement learning, Feedback controller, Guiding deep Q network, Dynamic obstacle avoidance

2024 IEEE 13th Data Driven Control and Learning Systems Conference (DDCLS) | 979-8-3503-6167-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/DDCLS61622.2024.10606847

1 Introduction

As research in robotics advances, an increasing number of robots are being utilized in the realm of industrial automation. Among these, robotic arms, which have found applications across various industrial settings, are particularly noteworthy. In factories, one can often observe numerous instances where robotic arms operate in tandem with conveyor belts, both of which have become indispensable components of automated production workshops. The robotic arms undertake tasks such as loading and unloading, welding, and spraying, while the conveyor belts handle transportation along extended assembly lines. A crucial step in the operation of a robotic arm, whether it is performing a pick-and-place task or otherwise, is the planning of a path from its initial position to the target location. This path must circumvent any potential obstacles within the workspace, a process referred to as obstacle avoidance path planning for the robotic arm. When a robotic arm operates in an environment with a conveyor belt, and the target object is not a workpiece on the belt, the workpiece can also influence the arm's obstacle avoidance path planning. For instance, when attempting to pick up an object located at a significant distance on the conveyor belt, moving workpieces on the belt can become dynamic obstacles in the arm's obstacle avoidance planning. Therefore, it is essential for the robotic arm to possess the capability to evade dynamic obstacles, necessitating the implementation of dynamic obstacle avoidance planning for the robotic arm.

Compared to mobile robots, robotic arm obstacle avoidance planning is significantly more challenging due to the high coupling between their links, resulting in limited research in this area. To address the dynamic obstacle avoidance problem of real robotic arms, Xie Long proposed a dynamic obstacle avoidance planning method based on an improved potential field method [1]. While this method can achieve dynamic obstacle avoidance in certain scenarios, it suffers from the drawback of easily generating local minima, necessitating the introduction of virtual obstacles or other algorithms to assist the robotic arm in escaping from these minima. Han et al. proposed a method that calculates the minimum distance between obstacles and the robotic arm to plan the obstacle avoidance path [2]. When the nearest distance is less than the safety value, collision detection is used to determine whether there are obstacles in the path planned by the robotic arm. If an obstacle is detected that will collide with the robotic arm, the path is replanned until the task is completed. However, this method requires multiple path planning iterations, which places high demands on the real-time performance of the path planning algorithm. Hongyan Liu et al. proposed a method that utilizes the complementary advantages of potential field and optimization based on quadratic programming to generate collision-free trajectories in real time [3]. However, this method requires offline planning of a relatively suitable collision-free path before utilizing real-time environmental information to optimize the trajectory for obstacle avoidance. Rapidly Exploring Random Tree (RRT) has been widely used in robotic arm path planning [4]. However, existing RRT algorithms have limitations in dynamic and unstructured environments. To address this, Duguleina et al. improved the RRT algorithm and applied it to dynamic obstacle avoidance for robotic arms, achieving positive results [5]. Their work highlights the ongoing research efforts towards developing more robust and effective solutions for robotic arm obstacle avoidance in complex scenarios.

In dynamic environments, existing path planning methods, demanding extensive online computation and prior knowledge, struggle to adapt to real-time changes, especially in complex settings. Deep reinforcement learning (DRL) offers a promising alternative, tackling challenging

*This work is supported by the Open Research Project of the State Key Laboratory of Industrial Control Technology, Zhejiang University, China (No.ICT2023B49).

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 04,2026 at 14:24:51 UTC from IEEE Xplore. Restrictions apply.

tasks traditionally difficult for conventional methods. Several DRL solutions have been proposed for dynamic obstacle avoidance. Y. Long and H. He successfully applied Deep Q Network (DQN) to mobile robot navigation [6]. S. Xiaoxian et al. developed a DRL-based obstacle avoidance algorithm for mobile robots considering obstacle interactions [7]. In the aerial realm, C. Sampedro et al. proposed a DDPG-based visual servo controller for multi-rotor UAVs [8]. DRL holds significant potential for robotic arm obstacle avoidance as well. Chen P. et al. introduced a DRL method using Soft Actor-Critic (SAC) for robot arm path planning, incorporating a combined reward function for obstacle avoidance and target approximation [9]. They further adopted prioritized experience replay (PER) to improve sample utilization. However, their method's reliance on AABB bounding boxes, overly conservative for non-cuboid objects, might limit flexibility in tight spaces. Another approach by T. Johannink et al. combines traditional feedback control with DRL through residual reinforcement learning for assembly tasks [10]. This offers advantages when traditional control aligns with task requirements, but increases training difficulty when they contradict.

This paper proposes a dynamic obstacle avoidance method based on Deep Q-Network (DQN) for a rigid robotic arm with six degrees of freedom. The state description of the robotic arm and the environment is simplified by using the keypoint method. Under this state description, a collision detection method for the deep reinforcement learning algorithm is defined, and a phased incremental reward value calculation method is used. A heterogeneous Guiding Deep Q Network (GDQN) is employed to help the actual network maintain a sufficient proportion of samples during training. Meanwhile, to prevent the robotic arm from making too many bad and meaningless explorations during the environmental exploration stage, a traditional feedback control algorithm designed by humans is combined with GDQN to help accumulate good experience and accelerate training. After training with the proposed method, the neural network can output the next operation strategy in real time according to the current state of the robotic arm and the environment to avoid dynamic obstacles and reach the target position.

2 Algorithm Principle

2.1 Reinforcement Learning

In a Markov decision process, an agent takes actions in a state at each time step, receives rewards, and probabilistically transitions to the next state based on the environment. Reinforcement learning refers to learning the mapping from environment states to actions. The agent learns the optimal policy by interacting with the environment through actions, receiving new state and reward information from the environment, and learning from these interactions and their outcomes. The goal is to maximize the expected return that the agent receives from the environment.[11]

2.2 Deep Q Network (DQN)

Q-learning is a value-based reinforcement learning algorithm. The Q in the algorithm represents the expected reward that can be obtained by taking an action at a given time in a given state. The environment will provide feedback to the agent's actions in the form of rewards, so the main idea of the algorithm is to construct a Q-table to store the Q-values for all possible state-action pairs. The agent then selects the action that has the highest Q-value for the current state. The initial values in the Q-table are randomly generated. Through continuous interaction with the environment, the agent explores the environment using a policy and can obtain the reward values for taking different actions in many states. The data obtained from these interactions is used to update the values in the Q-table, as shown in (1).

$$Q_{new}(s,a) = Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right] \quad (1)$$

α is learning rate, γ is discount factor and s' is the next state.

DQN is a reinforcement learning algorithm that uses deep learning to approximate the Q-function. It is typically used in discrete action spaces. DQN uses a neural network to replace the Q-table in Q-learning, and then selects the action that can obtain the maximum reward based on the Q-value output by the neural network. There are two neural networks with the same structure in the DQN algorithm: the prediction network and the target network. The prediction network is used to output the Q-values of all actions corresponding to a state, and this is used as the basis for selecting actions. The target network is used to assist the training of the prediction network, and produces a target function as shown in (2).

$$Target\ Q = r + \gamma \max_{a'} Q(s',a';\theta^{-}) \quad (2)$$

a' represents the action taken in the next state, and θ^{-} represents the parameters of the target network. Based on the $Target\ Q$, the loss function can be obtained as shown in (3).

$$Loss(\theta) = [Target\ Q - Q(s,a;\theta)]^2 \quad (3)$$

The DQN algorithm uses the loss function to update the parameters of the prediction network using gradient descent. The target network is then updated based on the prediction network.

2.3 Guiding Deep Q Network (GDQN)

DQN randomly explores the environment in the early stages of training and stores the experience samples generated during exploration in an experience pool, which is used to randomly extract samples for training in the later stages. However, random exploration can generate many low-quality policies, which can lead to poor training results. To address this issue, some people have proposed the Demonstrational DQN [12], which prepares an experience pool containing correct experience and samples a certain amount of experience from the pool in subsequent training, which is merged with random experience for network training. However, this method has high requirements for sample collection, and it is difficult to obtain a certain amount of correct experience in complex environments. Others have proposed the Double Experience Pool [13], which divides the samples generated by the agent's interaction with the environment into good and bad samples, and stores them in two separate experience pools. During training, samples are drawn from the two pools in a certain proportion. However, since the update speeds of the two experience pools cannot be synchronized, it is usually difficult to maintain a fixed sample ratio.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 04,2026 at 14:24:51 UTC from IEEE Xplore. Restrictions apply.

To address the aforementioned issues, this paper proposes to pre-train a guiding network before the actual network training in practical applications. The guiding network helps the actual network obtain high-quality samples in the early stage of training. To make the guiding network easier to converge, redundant state information can be used in the guiding network, and the network structure can be simplified. In this article, key points are used to describe the robotic arm instead of joint angles. Although in fact joint angles can infer the coordinates of key points, using key points directly as states can be more conducive to calculations and the identification of obstacles and target points. distance, can converge better. When the guiding network has basically achieved its function but there are still some defects in a small number of actions, the reward function can be adjusted in the actual network to train a better action selection strategy. The pseudo-code of the algorithm is shown in Table 1.

Table 1: Pseudocode of GDQN

Input: Q network $Q(s,a|\theta^Q)$, learnd guiding Q network $Q_g(s,a|\theta^{Q_g})$, target network Q' and ${Q_g}'$, feedback controllers: $\mu_1(s),\mu_2(s)$, replay buffer R
Output: learnd policy $Q(s,a|\theta^Q)$

Initialize $\varepsilon = 0.5, \varepsilon_g = 0.7$;
for $episode = 1, M$ **do**
  Get initial observation s_1;
  for $k = 1, K$ **do**
    if $random(0,1) < \varepsilon_g$ **then**
      if $random(0,1) < \varepsilon$ **then**
        Get random a_t;
      else
        $a_t = \max_a Q_g(s,a|\theta^{Q_g})$;
    else
      $a_t = \max_a Q(s,a|\theta^Q)$;
    Execute action a_t, observe reward r_t and next state s_{t+1};
    Store transition (s_t,a_t,r_t,s_{t+1}) in R;
    Sample a random minibatch of N transitions (s_t,a_t,r_t,s_{t+1}) from R;
    Get y_t according to Equation (2);
    Update $Q(s,a|\theta^Q)$ according to Equation (3);
    Update the target network Q';

3 Methods

In general, the actions taken by the robot arm to avoid obstacles will cause it to move away from the target. In RL, if no constraints are imposed on the exploration of the robot arm, it is very likely that the robot arm will exceed the safe workspace. Therefore, in position servo, a human-designed feedback controller is added to the robot arm. In this paper, based on the distance error between the end effector and the target, proportional control is used to enable the robot arm to approach the target, rather than exploring meaninglessly in the environment.

In residual reinforcement learning, the actions output by the feedback controller and the actions output by the policy network are usually directly added to obtain the final action. However, in this paper, the task of the robot arm is dynamic obstacle avoidance and position servo. The actions taken during obstacle avoidance are actually contradictory to the actions of the feedback controller. If the residual reinforcement learning method is directly used, it will be disadvantageous. At the same time, since the actions in the DQN algorithm are discrete, the proportional control coefficient of the feedback controller needs to be carefully considered in residual reinforcement learning.

G. Wang et al. used the critic network to evaluate the output signal of the base controller in the DDPG algorithm, thereby learning the grabbing and placing tasks of the robotic arm while only setting sparse rewards[14]. Therefore, the actions of the feedback controller are also added into the action space. The network is used to evaluate the quality of the actions and select the action with the highest evaluation. This action may come from the policy network itself, or it may come from the feedback controller. First, the feedback controller is combined with the guiding network to help with training, as shown in Figure 1.

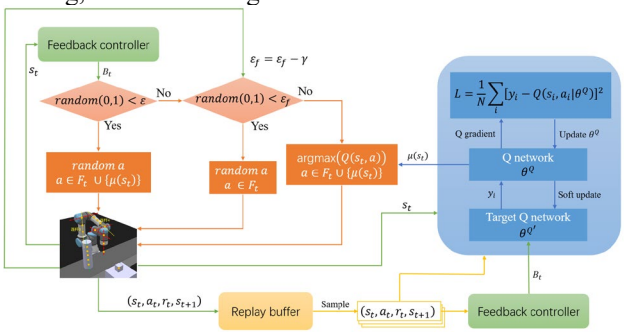
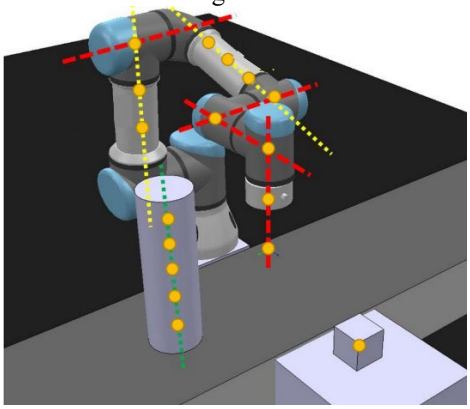


Fig. 1: DQN with feedback controller

3.1 State Space

3.1.1 Guiding Network State Space

To facilitate the training of the guiding network, this paper uses key points to describe the spatial positions of the robot arm, obstacles, and targets. The number of key points should be as small as possible, but the sphere with these key points as centers should be sufficient to envelop the object. This can more accurately describe the object state without occupying too much feasible space. The overall distribution of key points is shown in Figure 2.


Fig. 2: Distribution of key points for robot, obstacles and target

Let the key point on the robot be $p_{r_i}(i = 1,2,\cdots,10)$, the key point on the end effector be p_e, the key point on the obstacle be $p_{o_j}(j = 1,2,\cdots,5)$, and the key point on the target be p_t. The coordinates of each point in the base coordinate are:

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 04,2026 at 14:24:51 UTC from IEEE Xplore. Restrictions apply.

$$p_{r_i} = [x_{r_i}, y_{r_i}, z_{r_i}]$$
$$p_{o_j} = [x_{o_j}, y_{o_j}, z_{o_j}] \quad (4)$$
$$p_t = [x_t, y_t, z_t]$$
$$p_e = [x_e, y_e, z_e]$$

combine the coordinates of the above key points as the state space:

$$s = [s_r, s_o, s_t, s_e] \quad (5)$$

where s_r, s_o, s_t, s_e are the coordinate vectors of key points on the robot, obstacle, target and end effector respectively.

3.1.2 Actual Network State Space

In fact, the positions of p_{r_i} and p_e can be represented by the joint angles of the robot arm due to the coupling relationship between the links. Meanwhile, the key points of the obstacle are evenly distributed on the axis of the obstacle, so the positions of p_{o_j} can be determined by the base coordinate and height of the obstacle. Therefore, the joint angles of the robot arm, the base coordinate and height of the obstacle are used to replace the above key point coordinates.

$$s_{real} = [\Theta, p_o, p_t, h_o] \quad (6)$$

where $\Theta = [\theta_1, \theta_2, \cdots, \theta_6]$ represents the joint angle of the robot, p_o is the coordinate of the obstacle base point, h_o is the height of the obstacle, and p_t is still the coordinate of the target key point.

3.2 Action Space

3.2.1 Q Network Actions

The action space is selected as the fixed-step translation motion of the robot arm end effector in the Cartesian space along the X, Y, and Z axes and a keep-still action, with a total of 7 dimensions.

3.2.2 Feedback controller Actions

In this paper, two feedback controllers are set, one with high precision but small step length and the other with low precision but large step length. The proportional coefficients are 0.05 and 0.2 respectively. The robot arm will randomly select a controller from the two controllers to output the control signal. The feedback controllers are all 3-dimensional, representing the translation motion along the X, Y, and Z axes respectively. Therefore, the action space has a total of 13 dimensions. The robot arm will select an action from the feedback controllers with a probability of ε_f. As the number of training steps increases, ε_f will decrease by γ each time. Meanwhile, the robot arm also has a probability of $1 - \varepsilon_f$ to select an action with the highest Q-value from the 13 actions, as shown in Figure 1.

3.3 Reward Function

The reward mainly includes two parts: the reward related to the distance between the robot arm and the obstacle and the reward related to the distance between the end of the robot and the target.

The Euclidean distance between each key point p_{r_i} of the robot arm and the key point p_{o_j} of the obstacle is denoted by l_{ij}, and the reward function related to the obstacle is constructed as follows:

$$R_{obs} = \sum_{i=1}^{10} \sum_{j=1}^{5} \left(\frac{K_o}{max(l_{ij} - K, c_o)} \right)^2 \quad (7)$$

where K_o and K are constants, and c_o is a constant which is used to prevent the denominator from being zero.

The Euclidean distance between the end effector key point p_e and the target key point p_t is denoted by l_{tar}, and the reward function related to the target is constructed as follows:

$$R_{tar} = \frac{K_t}{max(l_{tar}, l_{suc}) + c_t} - h_t max(l_{tar}, l_{suc}) \quad (8)$$

where K_t, h_t, and c_t are constants, and l_{suc} is the distance threshold for determining whether the end effector key point has reached the target point. Within this threshold, it means that the target position has been reached, and the reward value does not need to be calculated based on l_{tar}, but is calculated based on l_{suc}.

Equations (7) and (8) together constitute the reward value of the current state.

$$R_{state} = R_{obs} + R_{tar} \quad (9)$$

In addition to the above dense rewards, there is also a sparse reward value that is used to reward when the robotic arm reaches the target position and does not collide with obstacles.

$$R_{suc} = \begin{cases} R_s & if\ l_{tar} < l_{suc}\ and\ min(l_{ij}) > l_{safe} \\ 0 & else \end{cases} \quad (10)$$

Since some actions have the same movement direction but slightly different distances, incremental reward values need to be used to evaluate the merits of each action. Assume that the state reward value at time t is R_{state}^t, and the state reward value at time $t+1$ after taking action a is R_{state}^{t+1}, then the reward value R obtained by this action is:

$$R = R_{state}^{t+1} - R_{state}^t + R_{suc} \quad (11)$$

In addition, when the robotic arm exceeds the working space or collides with an obstacle, a large negative reward value R_{fail} will be given instead of Equation (11).

4 Experiments

We first combines the feedback controller in the guiding network and uses the method shown in Figure 1 for training to obtain the guiding network $Q_g(s, a|\theta^{Q_g})$, and then uses Algorithm 1 when training the actual network.

4.1 Guiding Network

We present a simulation experiment conducted in CoppeliaSim using a UR3 robot. The objective of the robotic arm is to approach a small square in front of it while avoiding dynamic obstacles on a conveyor belt. In this environment, the conveyor belt continuously carries obstacles along the positive Y-axis, as shown in Figure 3.

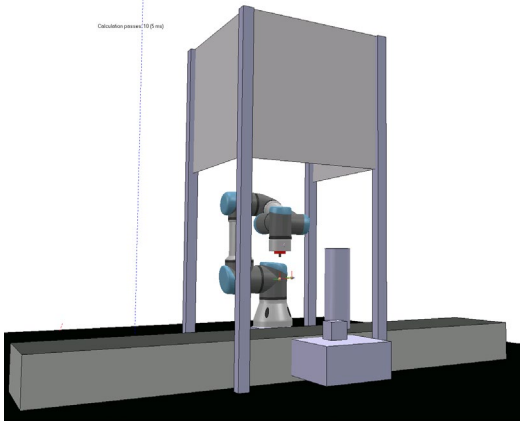


Fig. 3: Simulation training environment

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 04,2026 at 14:24:51 UTC from IEEE Xplore. Restrictions apply.

The guiding network adopts the DQN algorithm with an $\varepsilon - greedy$ strategy to explore the environment. The initial value of ε is set to 0.5. After 200 episodes, ε is multiplied by a decay factor of 0.9 every 10 episodes. The learning rate, initially set to 0.0007, is also multiplied by a decay factor of 0.99. As shown in Figure 4, the number of successful steps increases overall with the increase in training episodes. Although the feedback controller helps the robot approach the target quickly, it cannot avoid obstacles and collides before reaching the target position. When the robot collides, the training starts from a new episode, resulting in a low number of successful steps. The blue line shows the training result with the feedback controller, and the red line shows the training result without the feedback controller. As can be seen from the figure, the feedback controller can help the agent converge quickly.

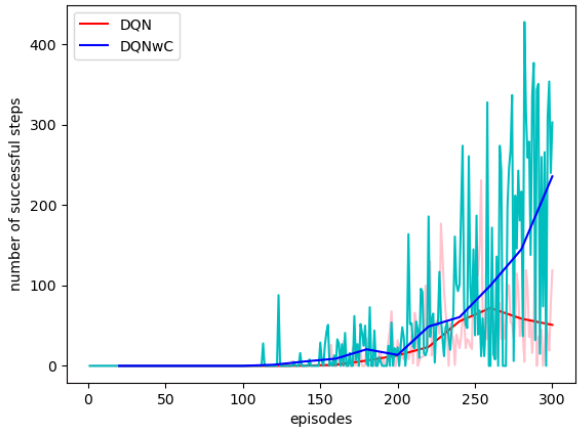


Fig. 4: The number of successful steps for each episode in guiding network training

The test results after training are shown in Figure 5, where the red line represents the minimum distance to the obstacle (l_{obs}) and the green line represents the distance to the target position (l_{tar}). When the obstacle approaches, the robot arm will move away from the target position to avoid the obstacle, and when the obstacle moves away, the robot arm will return to the target position. The whole obstacle avoidance process is shown in Figure 6.

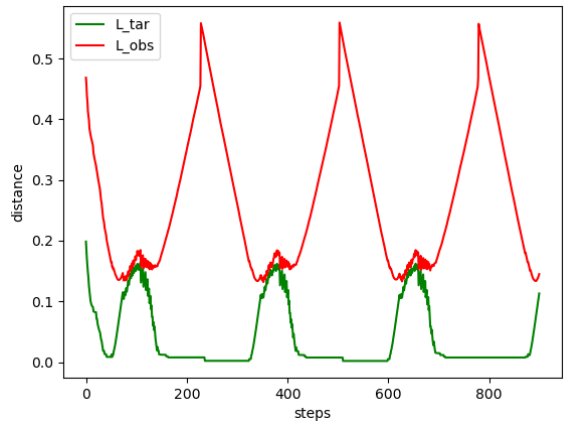


Fig. 5: Changes of l_{tar} and l_{obs} in the guiding network during the test round

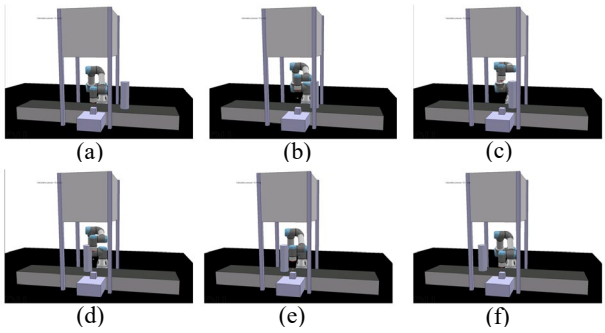


Fig. 6: An obstacle avoidance process of guiding network

4.2 Actual Network

The actual network and the guiding network are trained in the same simulation environment, and the guiding network is used to accumulate good samples.

The GDQN also adopts the $\varepsilon - greedy$ strategy, but this time it no longer needs a random strategy. Since the guiding network has already trained out good samples, there is a $1 - \varepsilon$ probability of selecting the action with the largest Q-value in the actual network output, and a ε probability of selecting the action with the largest Q-value in the guiding network output. The initial value of ε is 0.7. When the experience pool is full, the training starts, and ε is set to 0.5. After 300 episodes, ε is multiplied by a decay factor of 0.97 every 20 episodes. The learning rate still adopts the variable learning rate method, with an initial value of 0.004. After 300 episodes, the learning rate is multiplied by a decay factor of 0.99 every 20 episodes.

When the network structure, reward function, and hyperparameters are the same, the GDQN is compared with the DQN, as shown in Figures 7. We can see that the DQN has not learned how to complete this task.

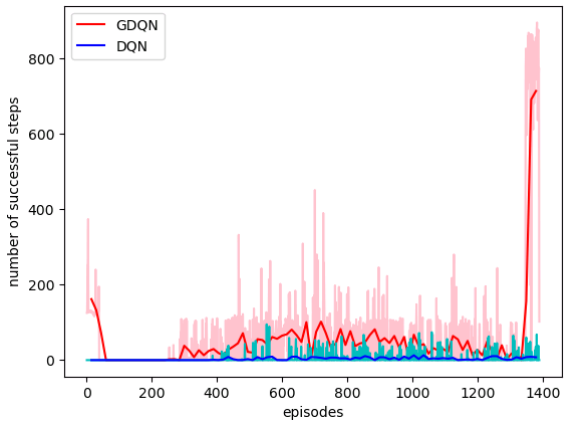


Fig. 7: The number of successful steps for each episode in actual network training

The test results after training are shown in Figure 8. Comparing Figure 5 and Figure 8, it can be seen that in guiding network, when the obstacle approaches, the obstacle avoidance actions of the robot arm have obvious oscillations, and the distance from the target object is also farther, and these issues are all improved in actual network. The whole obstacle avoidance process is shown in Figure 9.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 04,2026 at 14:24:51 UTC from IEEE Xplore. Restrictions apply.

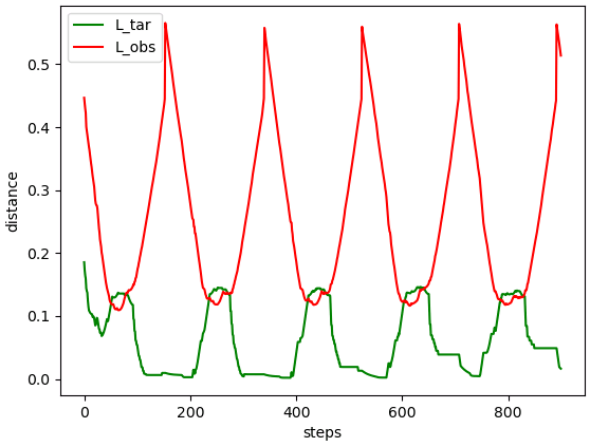


Fig. 8: Changes of l_{tar} and l_{obs} in the actual network during the test round

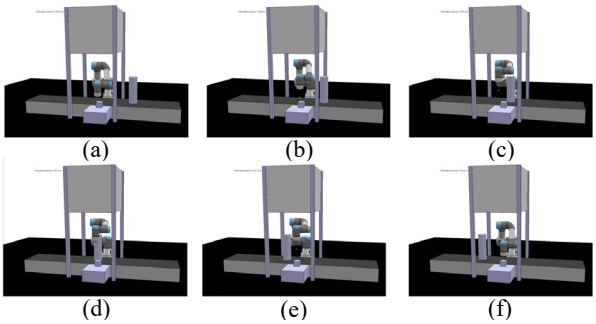


Fig. 9: An obstacle avoidance process of actual network

5 Conclusions

This paper proposes an improved residual deep reinforcement learning method for dynamic obstacle avoidance and robot arm position servo. Due to the coupling of the robot arm's links, the factors that need to be considered when avoiding obstacles are complex. We first simplify the state representation by establishing key points, and then use these states to pre-train a model that can complete the obstacle avoidance task. Subsequently, when using the actual state information such as the joint angles of the robot arm, the model of the guiding network is used to help accumulate good samples, so that the training can converge better and faster. However, even with the simplified state space, it is still difficult to converge. Therefore, based on residual reinforcement learning, the action of the feedback controller is incorporated into the action space to evaluate the pros and cons of the action. Otherwise, it is not conducive to the obstacle avoidance of the robot arm to apply the idea of residual reinforcement learning. A way of thinking is to use the network to choose whether to use the feedback controller. Since the original control signal of the feedback controller has repetition in direction but different in moving step length, this paper uses incremental reward values to evaluate the pros and cons of taking each action, and it can be used to correctly select actions. Finally, the feasibility of this method is verified in simulation.

References

[1] Xie L, Liu S, Dynamic obstacle-avoiding motion planning for manipulator based on improved artificial potencial field, Control Theory & Applications, 35(9):1230-1249, 2018.

[2] Han D, Nie H, Chen J, et al. Dynamic obstacle avoidance for manipulators using distance calculation and discrete detection[J]. Robotics and Computer-Integrated Manufacturing, 2018, 49: 98–104.

[3] Chen P, Pei J, Lu W, et al. A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance[J].Neurocomputing, 2022, 497: 64–75.

[4] Kuffner J J, LaValle S M. RRT-connect: An efficient approach to single-query path planning[C]//Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings. IEEE, 2000, 2: 995-1001.

[5] Duguleana M, Barbuceanu F G, Teirelbar A, et al. Obstacle avoidance of redundant manipulators using neural networks based reinforcement learning[J]. Robotics and Computer-Integrated Manufacturing, 2012, 28(2): 132-146.

[6] Long Y, He H. Robot path planning based on deep reinforcement learning[C]//2020 IEEE Conference on Telecommunications, Optics and Computer Science (TOCS). IEEE, 2020: 151-154.

[7] Xiaoxian SUN, Chenpeng YAO, Haoran Z, et al. Obstacle Avoidance Algorithm for Mobile Robot Based on Deep Reinforcement Learning in Dynamic Environments[C]//2020 IEEE 16th International Conference on Control & Automation (ICCA). IEEE, 2020: 366-372.

[8] Sampedro C, Rodriguez-Ramos A, Gil I, et al. Image-based visual servoing controller for multirotor aerial robots using deep reinforcement learning[C]//2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2018: 979-986.

[9] Chen P, Pei J, Lu W, et al. A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance[J]. Neurocomputing, 2022, 497: 64-75.

[10] Johannink T, Bahl S, Nair A, et al. Residual reinforcement learning for robot control[C]//2019 International Conference on Robotics and Automation (ICRA). IEEE, 2019: 6023-6029.

[11] Gao Y, Chen S F, Lu X. Research on reinforcement learning technology: a review[J]. ACTA AUTOMATICA SINICA. 2004, 30(001): 86-100.

[12] Hester T, Vecerik M, Pietquin O, et al. Deep q-learning from demonstrations[C]//Proceedings of the AAAI conference on artificial intelligence. 2018, 32(1).

[13] Huang Y, Liu D, Liu Z, et al. A novel robotic grasping method for moving objects based on multi-agent deep reinforcement learning[J]. Robotics and Computer-Integrated Manufacturing, 2024, 86: 102644.

[14] Wang G, Xin M, Wu W, et al. Learning of long-horizon sparse-reward robotic manipulator tasks with base controllers[J]. IEEE Transactions on Neural Networks and Learning Systems, 2022.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 04,2026 at 14:24:51 UTC from IEEE Xplore. Restrictions apply.